

Web Scraping + Email Notification System

Project Documentation - Books to Scrape

Project	book-data-scraping-analysis
Main Goal	Scrape book data and send email on success or failure
Language	Python
Libraries	Requests, BeautifulSoup, Pandas, python-dotenv
Email Method	Gmail SMTP
Output	books_data.csv + email notification

This document explains what was added to the project, how the files work together, how the .env file protects credentials, and how success/failure emails are sent.

1. Project Objective

The original project scraped book information from Books to Scrape and saved the data for analysis. We extended it so the scraping process can now report its result by email.

- **Success:** save the CSV and send an email showing that scraping completed.
- **Failure:** catch the error and send an email containing the error message.
- **Security:** keep the real Gmail address and App Password outside the Python source code.
- **Beginner-friendly:** use separate small Python files with simple responsibilities.

Overall Workflow

1	Run main.py
2	main.py calls scrape_books()
3	scraper.py visits all 50 pages
4	Book data is collected into a Pandas DataFrame
5	Data is saved to books_data.csv
6	Success email is sent
If error	Exception is caught and a failure email is attempted

Flow: Run Script -> Scrape Website -> Save CSV -> Send Success Email. If anything fails, the exception handler prepares a failure message and sends it.

2. Project File Structure

```
book-data-scraping-analysis/  
|  
|-- .env  
|-- .env.example  
|-- .gitignore  
|-- README.md  
|-- books_web_scraping.ipynb  
|-- email_notifier.py  
|-- main.py  
|-- requirements.txt  
|-- scraper.py  
`-- books_data.csv
```

What each file does

File	Purpose
books_web_scraping.ipynb	Original notebook for scraping, cleaning, analysis and visualization.
scraper.py	Performs the automated scraping and saves books_data.csv.
email_notifier.py	Reads email settings and sends email using Gmail SMTP.
main.py	Controls the complete process and handles success/failure.
.env	Stores real local email credentials. This file must not be uploaded to GitHub.
.env.example	Safe sample showing which environment variables are required.
.gitignore	Prevents .env and temporary files from being committed.
requirements.txt	Lists required Python packages.

3. scraper.py - Web Scraping Logic

This file contains the scraping function. It loops through pages 1 to 50, downloads each page, parses the HTML, extracts book information, and finally saves the collected records.

```
for page in range(1, 51):
    url = f"https://books.toscrape.com/catalogue/page-{page}.html"
    response = requests.get(url, timeout=15)
    response.raise_for_status()

    soup = BeautifulSoup(response.text, "html.parser")
```

requests.get() downloads the page. The **timeout=15** prevents the request from waiting forever.

response.raise_for_status() raises an error when the website returns an HTTP failure status. This is useful because main.py can catch that error.

BeautifulSoup converts the HTML into an object that is easier to search.

Data extracted from each book

Field	Example
Title	A Light in the Attic
Price	£51.77
Rating	3
Availability	In stock
Product URL	Full product page URL

```
df = pd.DataFrame(books)
df.to_csv("books_data.csv", index=False)

return len(df)
```

The function returns the number of scraped rows so main.py can include it in the success email.

4. email_notifier.py - Email Sending

This file is responsible only for sending email. Keeping email logic separate makes the project easier to understand and maintain.

```
import os
import smtplib
from email.message import EmailMessage
from dotenv import load_dotenv

load_dotenv()
```

load_dotenv() reads values from the local .env file and places them into environment variables for the Python process.

```
sender_email = os.getenv("EMAIL_ADDRESS")
app_password = os.getenv("EMAIL_APP_PASSWORD")
receiver_email = os.getenv("EMAIL_TO", sender_email)
```

The source code never contains the real credentials. Instead, it asks the operating environment for them.

```
with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
    smtp.login(sender_email, app_password)
    smtp.send_message(email)
```

Gmail SMTP over SSL uses port 465. The app logs in with the sender email and Google App Password, then sends the prepared message.

5. main.py - Success and Failure Handling

main.py is the controller. It runs the scraper inside a try/except block.

```
try:
    total_books = scrape_books()

    send_email(
        "Web Scraping Completed",
        success_message
    )

except Exception as error:
    send_email(
        "Web Scraping Failed",
        failure_message
    )
```

Why try/except is important

- Without try/except, the program would stop immediately after an error.
- With try/except, we can capture the error object.
- The error text can be included in the failure notification.
- This gives useful monitoring even when the user is not watching the terminal.

Situation	Result
Scraping completes	CSV is saved and a success email is sent.
Website request fails	Error is caught and a failure email is attempted.
Parsing code fails	Error is caught and a failure email is attempted.
Email sending fails	The secondary email error is printed in the terminal.

6. .env File and Security

The .env file is stored only on the local computer. It contains the real values needed by the email script.

```
EMAIL_ADDRESS=yourgmail@gmail.com
EMAIL_APP_PASSWORD=your_16_character_app_password
EMAIL_TO=yourgmail@gmail.com
```

Never upload the real .env file to GitHub. A password or token committed to a public repository may be copied by other people or automated scanners.

Why .env.example is safe

.env.example contains only placeholder values. It teaches another developer which variables must exist without exposing real secrets.

Why .gitignore matters

```
.ipynb_checkpoints/
__pycache__/*
*.pyc
.env
```

Because .env is listed in .gitignore, Git normally does not include it in commits.

Safety check before pushing

```
git status
```

If .env does not appear in the list of files to be committed, Git is ignoring it correctly.

7. How to Run the Project

Step 1 - Pull latest changes

```
git switch main  
git pull origin main
```

Step 2 - Install requirements

```
pip install -r requirements.txt  
pip install python-dotenv
```

Step 3 - Create .env locally

```
EMAIL_ADDRESS=yourgmail@gmail.com  
EMAIL_APP_PASSWORD=your_app_password  
EMAIL_TO=yourgmail@gmail.com
```

Step 4 - Run

```
python main.py
```

Expected success output

```
Scraping completed successfully.  
Total books scraped: 1000
```

Expected failure output

```
Scraping failed: <actual error>  
Failure email could not be sent: <email error> # only if email also fails
```

8. Example Email Notifications

Success email

Subject	Web Scraping Completed
Website	Books to Scrape
Records	1000
File	books_data.csv
Time	Current completion time

Failure email

Subject	Web Scraping Failed
Website	Books to Scrape
Error	Actual Python/network/HTTP error
Time	Current failure time

Common Problems

Problem	What to check
Email settings are missing	Check .env variable names and confirm load_dotenv() exists.
SMTPAuthenticationError	Check Gmail address and App Password; do not use the normal Gmail password.
App Password unavailable	Google account security settings may not allow App Passwords.
ModuleNotFoundError: dotenv	Run: pip install python-dotenv
Program takes time	The scraper visits 50 pages, so completion is not instant.
No email but CSV created	Scraping worked; investigate email configuration separately.

9. How to Explain This Project in an Interview

A simple explanation you can use:

"I built a Python web scraping project that collects book data from Books to Scrape using Requests and BeautifulSoup. I separated the scraping and email logic into different files. The scraper saves the result to CSV, and main.py uses exception handling to detect whether the process succeeds or fails. After the job finishes, the project sends a success or failure notification through Gmail SMTP. I used environment variables and a .env file so credentials are not stored in the source code or GitHub."

Important concepts demonstrated

- HTTP requests
- HTML parsing with BeautifulSoup
- Loops and dictionaries
- Pandas DataFrame and CSV export
- Functions and modules
- try/except exception handling
- SMTP email
- Environment variables
- Credential security using .gitignore
- Git and GitHub workflow

Possible Future Improvements

- Run automatically every day using Windows Task Scheduler or GitHub Actions.
- Attach the generated CSV to the success email.
- Store logs in a file.
- Scrape multiple websites.
- Add retry logic for temporary network failures.
- Use an email API if Gmail App Passwords are unavailable.

Final Result

The project is no longer only a notebook-based scraper. It now has a small automation workflow that can run independently, save data, handle errors, and notify the user by email.